Laboratory Practice III

Oral / Viva Questions with Short Answers

SPPU – Fourth Year Computer Engineering (2019 Course)

This revision sheet covers the five Python practicals prepared in this conversation: Fibonacci, Huffman Encoding, Fractional Knapsack, 0/1 Knapsack and N-Queens.

1. Fibonacci Series – Recursive and Non-Recursive

Q1. What is the Fibonacci series?

Answer: A sequence in which each term is the sum of the two previous terms: 0, 1, 1, 2, 3, 5, and so on.

Q2. What is recursion?

Answer: Recursion is a technique in which a function calls itself to solve a smaller version of a problem.

Q3. What is iteration?

Answer: Iteration means repeating a set of statements using a loop, such as a for or while loop.

Q4. What are the base cases in recursive Fibonacci?

Answer: The base cases are F(0) = 0 and F(1) = 1.

Q5. What happens if a recursive function has no base case?

Answer: It may continue calling itself until Python raises a RecursionError or the recursion limit is reached.

Q6. What is the time complexity of the iterative Fibonacci program?

Answer: O(n), because it calculates n terms using one loop.

Q7. What is the space complexity of the iterative version?

Answer: O(1), because it uses only a fixed number of extra variables.

Q8. What is the time complexity of the simple recursive Fibonacci program?

Answer: O(2^n) as a commonly used upper-bound estimate, because many subproblems are recalculated.

Q9. What is the auxiliary space complexity of recursive Fibonacci?

Answer: O(n), due to the maximum recursion call stack depth.

Q10. Which method is more efficient for calculating many Fibonacci terms?

Answer: The iterative method is generally more efficient than the simple recursive method.

2. Huffman Encoding – Greedy Strategy

Q1. What is Huffman Encoding?

Answer: It is a lossless data compression technique that assigns variable-length binary codes to characters.

Q2. What is the main idea of Huffman Encoding?

Answer: Characters with higher frequencies get shorter codes, while less frequent characters get longer codes.

Q3. Which algorithmic strategy does Huffman Encoding use?

Answer: It uses the greedy strategy.

Q4. What is a greedy strategy?

Answer: A strategy that makes the best local choice at each step, aiming to build an overall optimal solution.

Q5. What data structure is commonly used in Huffman Encoding?

Answer: A min-heap or priority queue.

Q6. Why is a min-heap used?

Answer: It efficiently retrieves the two nodes with the smallest frequencies.

Q7. What happens at each step of Huffman tree construction?

Answer: The two nodes with the lowest frequencies are removed, combined into a parent node, and inserted back into the heap.

Q8. What do 0 and 1 represent in the Huffman tree?

Answer: They represent the left and right branches used to build each character's binary code.

Q9. What is a prefix-free code?

Answer: A code in which no character's code is the prefix of another character's code.

Q10. What is the time complexity of Huffman Encoding using a heap?

Answer: O(n log n), where n is the number of distinct characters.

3. Fractional Knapsack – Greedy Method

Q1. What is the Knapsack problem?

Answer: It is an optimisation problem in which items with weights and profits are selected without exceeding a given capacity.

Q2. What is Fractional Knapsack?

Answer: A version of the Knapsack problem in which an item can be divided and a fraction of it can be selected.

Q3. Which strategy is used to solve Fractional Knapsack?

Answer: The greedy strategy.

Q4. How do you choose items in Fractional Knapsack?

Answer: Calculate profit-to-weight ratio for each item and choose items in descending order of this ratio.

Q5. What is the profit-to-weight ratio?

Answer: It is the profit of an item divided by its weight.

Q6. Why is sorting required?

Answer: Sorting helps select the item with the highest profit per unit of weight first.

Q7. Can an item be partially selected?

Answer: Yes. If the remaining capacity is smaller than the item's weight, a fraction of that item is selected.

Q8. What is the difference between Fractional and 0/1 Knapsack?

Answer: Fractional Knapsack allows splitting items; 0/1 Knapsack requires taking an item completely or not taking it.

Q9. What is the time complexity of the greedy Fractional Knapsack solution?

Answer: O(n log n), mainly due to sorting the items.

Q10. Does the greedy method give an optimal solution for Fractional Knapsack?

Answer: Yes, selecting by descending profit-to-weight ratio gives an optimal solution for the standard Fractional Knapsack problem.

4. 0/1 Knapsack – Dynamic Programming

Q1. What is 0/1 Knapsack?

Answer: A problem where each item can either be selected once or not selected, subject to the capacity limit.

Q2. Why is it called 0/1 Knapsack?

Answer: Because the decision for each item is binary: 0 means not selected and 1 means selected.

Q3. Which technique is used in this program?

Answer: Dynamic Programming.

Q4. What is Dynamic Programming?

Answer: A method that solves a problem by storing solutions to smaller overlapping subproblems and reusing them.

Q5. What does dp[i][w] represent?

Answer: The maximum profit using the first i items with a knapsack capacity w.

Q6. What are the two choices for each item?

Answer: Include the item if it fits, or exclude the item.

Q7. What is the recurrence relation when an item fits?

Answer: dp[i][w] = max(profit[i-1] + dp[i-1][w-weight[i-1]], dp[i-1][w]).

Q8. What happens when an item is heavier than the current capacity?

Answer: The item cannot be included, so the previous row's value is copied.

Q9. What is the time complexity of the standard DP solution?

Answer: O(nW), where n is the number of items and W is the capacity.

Q10. What is the space complexity of the 2D DP solution?

Answer: O(nW), because it stores a table of (n+1) by (W+1) values.

Q11. What is the key difference between Fractional and 0/1 Knapsack?

Answer: Fractional Knapsack permits partial items and is solved greedily; 0/1 Knapsack does not permit splitting and can be solved using Dynamic Programming.

5. N-Queens – Backtracking

Q1. What is the N-Queens problem?

Answer: The task is to place N queens on an N by N chessboard so that no two queens attack each other.

Q2. What is backtracking?

Answer: A problem-solving method that builds a solution step by step and reverses a choice when it cannot lead to a valid solution.

Q3. Why is backtracking used in N-Queens?

Answer: It tries possible queen positions and removes a queen when the placement makes a valid solution impossible.

Q4. How can a queen attack another queen?

Answer: A queen attacks along the same row, column, or diagonal.

Q5. What does the is_safe() function check?

Answer: It checks whether placing a queen at a position conflicts with any previously placed queen.

Q6. Why does the program check previous rows?

Answer: Because the program places one queen per row, so only already-filled rows need to be checked.

Q7. Why is the first queen fixed at (0, 0) in this practical?

Answer: The problem statement asks to place the first queen before using backtracking for the remaining queens.

Q8. What does the solve() function do?

Answer: It tries safe columns in the current row, places a queen, and recursively solves the next row.

Q9. Why is a queen removed by setting the board cell back to 0?

Answer: This undoes a failed choice so the algorithm can try another column.

Q10. What does the base case row == n mean?

Answer: All N queens have been placed successfully.

Q11. What is the time complexity commonly stated for N-Queens backtracking?

Answer: O(N!) as a commonly used upper-bound estimate for the search.

Q12. What is the space complexity of the board representation?

Answer: $O(N^2)$ for the board, plus O(N) recursion stack space.

Quick Complexity Revision

Fibonacci: Iterative O(n) time, O(1) extra space; simple recursive O(2^n) time, O(n) stack space.
Huffman Encoding: O(n log n) time, O(n) space.
Fractional Knapsack: O(n log n) time, O(n) space for sorting/order storage.
0/1 Knapsack (2D DP): O(nW) time and O(nW) space.
N-Queens: commonly stated O(N!) search-time upper bound; $O(N^2)$ board space plus O(N) recursion stack.

Exam note: Complexity can vary with the exact implementation and the way auxiliary space is counted. Be ready to explain what n, W and N represent in each practical.